

Reporte Proyecto 1

Cabrera Galindo Daniela Crisel
Delgadillo Rodriguez Erika Gabriela
Martinez Flores Ashley
Rodriguez Becerril Alejandro

Septiembre de 2026

Modelado y Programacion

Profesor: Galaviz Casas Jose de Jesus
Ayudantes: Ayala Flores Brenda
Cruz Campos Jose Eduardo

1. Definicion del problema

El problema nos pide que hagamos un programa que, usando imagenes con terminacion .bmp, identifique cada figura presente en la imagen, decir su color en formato hexadecimal y clasificarla en una de las cuatro categorías que nos dan (C de cuadrilateros, T de triangulos, O de circulos y X de otros), donde si la imagen debe de tener una o más figuras de colores sólidos, distintos entre sí y distintos del fondo, sin degradados ni suavizado en los bordes, tambien debe de ser claro, eficiente y mantenible.

2. Arsenal

Elegimos a Python como el lenguaje que usariamos porque las librerias que tiene (como Pillow, NumPy, SciPy, etc.) nos permite leer archivos .bmp, la segmentación de regiones en las imagenes y el analisis de las figuras con pocas líneas de código y es muy claro, esto nos facilita experimentar con el codigo y lo que hace el codigo mantenible. Otras opciones que se vieron para usar fueron C++ y Java. Aunque C++ ofrece mayor rendimiento y por el lado de Java ofrece mayor portabilidad, para este caso el rendimiento no es un gran problema y la portabilidad ya está cubierta por usar Python, entonces el usar Python nos da muchas ventajas para este proyecto, además de que la sintaxis de Python es parecida al pseudocódigo, entonces nos facilita escribirlo en pseudocodigo en lugar de diagrama de flujo.

Se usaron tres librerías para el proyecto (OpenCV (cv2), NumPy y sys). Se uso OpenCV porque resuelve de forma eficiente la lectura de imágenes .bmp, la segmentación por color, la detección de contornos y el calculo del número de vértices y circularidad para el clasificador. Luego se uso NumPy porque es el formato en el que OpenCV representa las imágenes y tambien porque permite operar sobre los píxeles de forma vectorizada, y esto simplifica el cálculo del color en cada figura, además evita bucles. Y usamos sys porque es la manera mas portable de leer la ruta de la imagen desde la línea de comandos.

3. Analisis del problema

3.1. Supuestos

Para trabajar nuestro problema buscamos que el fondo de la imagen sea de un solo color uniforme, que cada figura tenga un color solido, que el color de cada figura sea distinto entre si y que el color del fondo no coincida con ninguno de los colores de las figuras, que no haya degradado en los bordes, que las figuras no se superpongan unas con otras, cada figura esta completamente dentro de la imagen y no solo una parte de esta y la imagen debe ser tipo .bmp.

3.2. Casos limite

En este caso los casos limites que vimos que podrian pasar son:

- Si la ruta sea no sea válida, que no sea un archivo .bmp o que no haya archivo, para este caso se deberia de mostrar un mensaje claro que aclare esto y avisar que no es una imagen válida.
- Si hay una imagen sin figuras se debe dar un mensaje diciendo que no se encontraron figuras.
- Si alguna figura toca o cruza el borde de la imagen se puede ignorarla y no contarla o clasificarla como X.
- Si alguna figuras es muy pequeña, es decir de pocos píxeles, pueden confundirse, o por el contrario si es muy grande puede afectar el rendiemento, entonces se puede aplicar un área mínima y un area maxima para identificar las figuras.
- Si hay cualquier tipo de figura irregular se clasificara como X, por ejemplo los cuadrilateros concavos.
- Si los colores de una figura son muy similares a los del fondo se puede confundir, entonces habria que usar tolerancia.
- Si dos figuras tienen el mismo color se deben considerar cada una por aparte.

3.3. Requisitos funcionales

En este caso los requisitos funcionales que buscamos serian:

- Recibir como entrada la ruta de una imagen .bmp.
- Validar que la ruta exista, que el archivo sea legible y que sea formato .bmp.
- Detectar el color de fondo e identificar las figuras, separándolas del fondo.
- Poder procesar imágenes con más de una figura.
- Reportar cuando no se encuentren figuras en la imagen.
- Para cada figura, determinar su color y convertirlo a formato hexadecimal.
- Clasificar cada figura en una de las categorías definidas.
- Mostrar al usuario un reporte claro con el color hexadecimal y la categoría de cada figura con mensajes claros y comprensibles.

3.4. Requisitos no funcionales

Ahora los requisitos no funcionales serian:

- Que los mensajes sean claros y comprensibles.
- Que tenga buen rendimiento, es decir que procese una imagen de tamaño razonable en un tiempo aceptable.
- Que sea robusto, es decir que no debe terminar abruptamente ante entradas inválidas y, al contrario debe de avisar.
- El codigo debe de tener nombres claros, funciones bien separadas y documentación suficiente.
- Debe de ser portable.
- Debe de ser preciso bajo los supuestos del proyecto.
- Debe de poder manejar varias figuras y distintos tamaños de imagen sin afectar gravemente su desempeño.

4. Selecccion de la mejor alternativa

Aqui veremos parte por parte del codigo para ver si es la mejor alternativa:

- **Array.py:** Para detectar y clasificar las figuras escogimos que primero segmentamos la imagen por color para obtener los contornos, luego filtramos los contornos por área mínima para descartar el ruido, luego aproximar cada contorno a un polígono con `cv2.approxPolyDP` usando un ϵ del 4% del perímetro, luego contamos los vértices y lo clasificamos y por ultimo obtenemos el color muestreando el píxel en el centroide calculado con momentos de imagen. Elegimos esta opcion sobre otras (como la transformada de Hough o los momentos de Hu) porque es determinista, explicable, eficiente y robusto para el proyecto, ya que trabajamos con colores sólidos, no hay anti-aliasing, no hay traslape y hay fondo uniforme.
Mientras, por otro lado tenemos que la transformada de Hough que solo detecta círculos y es costosa o los momentos de Hu que requiere de plantillas y es menos explicable. Todas estos obstaculos con los otros metodos no las hacen opciones tan optimas para el codigo.
- **Clasificacion.py:** para asignar la categoría a cada figura optamos por un clasificador basado en el número de vértices, así nuestro `clasificacion.py` recibe una figura, calcula sus "puntas" y devuelve una categoría. Elegimos este enfoque sobre alternativas como la clasificación por circularidad o los momentos de Hu porque es determinista, es explicable y esta perfectamente alineado con los supuestos del proyecto, es decir figuras geométricas limpias, no hay anti-aliasing y sin traslape.
Para este caso la clasificacion por circularidad requiere umbrales arbitrarios y no distinguiria bien entre las cuatro categorías o en el caso de los momentos de Hu requiere plantillas y es menos explicable. Además, el diseño que se escogio nos permite probar a `clasificacion_de_figuras` de forma aislada, sin necesidad de cargar imágenes, lo que mejora la testabilidad y la mantenibilidad del código.
- **Lector.py:** Para cargar las imagenes escogimos usar la lectura con `cv2.imread` validar su valor de retorno, es decir si devuelve un `None` (que es cuando la ruta no existe, el archivo está corrupto o el formato no es soportado) lanzamos un `FileNotFoundError` con un mensaje claro que indica al usuario que verifique la ruta y el formato. Elegimos esta opcion sobre implementar un parser manual de BMP o usar Pillow porque OpenCV ya nos da la lectura de `.bmp` de forma robusta, soporta múltiples variables del formato y ofrece las funciones geométricas que usa el resto del pipeline evitando añadir dependencias innecesarias, ahora se decidio dar el mensaje de error cuando da `None` por el requisito de robustez y usabilidad para que el programa no "true"ne con un error mas adelante.
Ahora para la conversión de color a hexadecimal usamos `{:02x}` con `.upper()` porque garantiza el relleno con cero a la izquierda, esto nos evita el hex mal formados como `#5A0` en lugar de `#050A00`, y produce un formato mas consistente y mas legible.
- **Main.py:** Para esta parte usamos un diseño donde `main.py` se encarga de la entrada, el preprocesamiento y la extracción de contornos, luego analiza y clasifica a cada figura, para segmentar las figuras primero convertimos la imagen a escala de grises con `cv2.cvtColor` aplicamos una umbralización con `cv2.threshold` usando un umbral fijo de 240. Elegimos esta opcion sobre la detección del color de fondo por moda de los bordes porque es más simple, más corto y más fácil de explicar, en este caso asumimos que el fondo es claro, es decir con una luminancia de mas de 240, y que las figuras son oscuras o saturadas, es decir con luminancia menor a 240, lo cual se cumple en nuestras imagenes.
El uso de `"THRESH_BINARY_INV"` es necesario porque `cv2.findContours` espera que las regiones blancas sean objetos, y para extraer los contornos usamos `RETR_EXTERNAL` y `CHAIN_APPROX_SIMPLE` el primero porque las figuras son sólidas y sin agujeros y el segundo porque reduce el número de puntos sin perder información. Por ultimo para los errores usamos una entrada con `"try/except FileNotFoundError"` validamos los argumentos con `"sys.argv"` para cumplir con la robustez.

5. Pseudocodigo

Ahora el pseudocodigo de nuestro codigo quedaria:

```
ALGORITMO ReconocimientoDeFiguras
ENTRADA: ruta_imagen (string) – ruta a un archivo .bmp
SALIDA: por cada figura detectada: categoría (C/T/O/X) y color en hexadecimal

INICIO
// ----- Bloque principal (main.py) -----
SI longitud(argv) ≠ 2 ENTONCES
    IMPRIMIR "Uso incorrecto. Ejecuta:"
    IMPRIMIR "python main.py ruta_de_la_imagen.bmp"
    TERMINAR
FIN SI

ruta ← argv[1]
procesar_imagen(ruta)
FIN

(Lector.py)

FUNCIÓN cargar_imagen(ruta_imagen):
    imagen ← cv2.imread(ruta_imagen, MODULO_COLOR)
    SI imagen = NULO ENTONCES
        LANZAR FileNotFoundError(
            "Error: No se pudo cargar la imagen '<ruta>'. "
            "Verifica la ruta y que sea .bmp"
        )
    FIN SI
    RETORNAR imagen
FIN FUNCIÓN
```

Figura 1: Imagen 1

```
FUNCIÓN convertir_a_grises(imagen_color):
    RETORNAR cv2.cvtColor(imagen_color, BGR_A_GDIS)
FIN FUNCIÓN

FUNCIÓN rgb_a_hexadecimal(rojo, verde, azul):
    // hex2(x) formatea x en hexadecimal con 2 dígitos y relleno con 0
    RETORNAR "#" + hex2(rojo) + hex2(verde) + hex2(azul) // en mayúsculas
FIN FUNCIÓN

(main.py)

FUNCIÓN procesar_imagen(ruta_imagen):

    // ----- Paso 1: Cargar imagen con manejo de errores -----
    INTENTAR
        imagen ← cargar_imagen(ruta_imagen)
    CAPTURAR FileNotFoundError COMO error
    IMPRIMIR error
    RETORNAR
    FIN INTENTAR

    // ----- Paso 2: Convertir a escala de grises -----
    gris ← convertir_a_grises(imagen)

    // ----- Paso 3: Umbralizar con binario invertido -----
    // Los píxeles con luminancia > 240 se consideran fondo (0),
    // los demás se consideran figura (255).
    (_, binarizada) ← cv2.threshold(gris, 240, 255, THRESH_BINARY_INV)
```

Figura 2: Imagen 2

```
// ----- Paso 4: Detectar contornos externos -----
(contornos, _) ← cv2.findContours(
    binarizada,
    RETR_EXTERNAL,
    CHAIN_APPROX_SIMPLE
)

// ----- Paso 5: Delegar el análisis a Array.py -----
analizar_arreglo_figuras(contornos, imagen)
FIN FUNCIÓN

(Array.py)

FUNCIÓN analizar_arreglo_figuras(contornos, imagen):

    // ----- Paso 1: Verificar que haya contornos -----
    SI contornos está vacío ENTONCES
        IMPRIMIR "No se detectaron figuras en la imagen."
        RETORNAR
    FIN SI

    // ----- Paso 2: Filtrar contornos por área mínima -----
    contornos_validos ← lista vacía
    PARA CADA contorno EN contornos HACER
        SI cv2.contourArea(contorno) > 100 ENTONCES
            agregar contorno a contornos_validos
        FIN SI
    FIN PARA

    IMPRIMIR "Se encontraron <n> figura(s). Analizando..."
    IMPRIMIR "-----"
```

Figura 3: Imagen 3

```
// ----- Paso 3: Procesar cada figura -----
PARA i DESDE 1 HASTA longitud(contornos_validos) HACER

    contorno ← contornos_validos[i]

    // 3.1 – Calcular perímetro y aproximar a polígono
    perimetro ← cv2.arclength(contorno, cerrado = VERDADERO)
    aproximacion ← cv2.approxPolyDP(contorno, 0.04 * perimetro, VERDADERO)
    puntas ← longitud(aproximacion)

    // 3.2 – Normalizar vértices para el clasificador
    SI puntas > 6 ENTONCES
        puntas ← 0
    FIN SI

    // 3.3 – Clasificar la figura
    categoria ← clasificacion_de_figuras(puntas)

    // 3.4 – Calcular centroide con momentos
    M ← cv2.moments(contorno)
    SI M["m00"] ≠ 0 ENTONCES
        cX ← entero(M["m10"] / M["m00"])
        cY ← entero(M["m01"] / M["m00"])
    SINO
        (cX, cY) ← contorno[0][0]
    FIN SI

    // 3.5 – Obtener color en el centroide (OpenCV usa BGR)
    color_bgr ← imagen[cY, cX]
    azul ← entero(color_bgr[0])
    verde ← entero(color_bgr[1])
    rojo ← entero(color_bgr[2])
```

Figura 4: Imagen 4

```
// 3.6 – Convertir a hexadecimal
hex_color ← rgb_a_hexadecimal(rojo, verde, azul)

// 3.7 – Reportar resultado
IMPRIMIR "Figura <i>:"
IMPRIMIR " - Categoría: <categoria>"
IMPRIMIR " - Color (Hex): <hex_color>"
IMPRIMIR "-----"

FIN PARA

FIN FUNCIÓN

(Clasificacion.py)

FUNCIÓN clasificacion_de_figuras(puntas):

    SI puntas = 0 ENTONCES
        RETORNAR "O" // círculo (o figura no poligonal)
    FIN SI

    SI puntas = 4 ENTONCES
        RETORNAR "C" // cuadrilátero
    FIN SI

    SI puntas = 3 ENTONCES
        RETORNAR "T" // triángulo
    FIN SI

    SI puntas < 3 O puntas > 4 ENTONCES
        RETORNAR "X" // otros
    FIN SI

FIN FUNCIÓN
```

Figura 5: Imagen 5